Lab 08 Storage SQL and secret protection

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a data-access and recovery exception report. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use py -3 if python3 is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- mock-data.json: synthetic input with no real personal data or credentials.
- analyse.py: runnable analysis for this lab only.
- expected-results.json: expected baseline and source checksum.
- ai-review-prompt.txt: evidence-constrained AI review prompt.
- evidence-template.md: your deliverable template.
- INSTRUCTIONS.md and INSTRUCTIONS.pdf: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open analyse.py in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open results.json. The expected decisions in output order are: BASELINE_OK, ACCESS_REVIEW, SECRET_EXPOSURE,RPO_BREACH. The source_sha256 binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-
results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Storage access boundaries

Data-plane roles, shared keys, SAS and public access create distinct access paths.

Contract: allow_public, allow_shared_key, sas_expiry, data_role

Accepted case: An identity receives scoped blob data access with public access disabled.

Counterexample: An account key enables broad access outside the intended identity scope.

Diagnosis: Inventory every enabled access path before calling a storage account private.

Evidence: Capture role scope, anonymous denial and expiry of temporary grants.

SQL and secret protection

A database authenticates a principal and authorises its operations; a vault limits secret retrieval.

Contract: sql_principal, database_role, vault_role, secret_version

Accepted case: A read-only database user and scoped secret reader serve a reporting workload.

Counterexample: A shared administrator secret is pasted into a notebook.

Diagnosis: Use an approved identity path and rotate any exposed live secret; keep values out of screenshots.

Evidence: Record role names and secret metadata, never the secret value.

Retention and recovery

Recovery points and restore tests establish recoverability independently of encryption.

Contract: backup_time, restore_time, rpo_minutes, rto_minutes

Accepted case: A 15-minute data gap meets a 30-minute RPO.

Counterexample: A backup exists but the restore fails due to missing key access.

Diagnosis: Test restoration in an isolated target and include key availability in the recovery design.

Evidence: Record measured data loss and elapsed restoration time.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Set data-c secret_in_code to false. RPO_BREACH must remain; access hygiene and recovery objectives are separate.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure F Storage SQL and vault controls

1. Open the assigned storage account > Configuration and Networking. Record anonymous/public and shared-key settings. Inspect data-plane role assignments separately from management roles.
2. Use the trainer-approved test identity to demonstrate one allowed data operation and one operation outside its intended scope. Do not list or copy account keys merely to obtain access.
3. Review a synthetic or trainer-approved SAS example. Identify service/resource scope, permissions and expiry. Do not place a live SAS URL in the evidence or an AI prompt.
4. Open the assigned Key Vault > Access configuration and identify the permission model. For Azure RBAC inspect Access control (IAM); for the legacy model inspect Access policies. Record the test identity's actual secret-access scope under the selected model. Create a synthetic training secret only if allocated; use a value such as TRAINING-ONLY-NOT-A-CREDENTIAL and never reuse a real secret.
5. Capture secret name/version metadata and an authorised or denied test, not the value. Remove the synthetic secret under trainer supervision after evidence capture.
6. Open the assigned Azure SQL server/database > Security. Review authentication configuration, network access and auditing. In an approved query session inspect the assigned database role without granting administrator access.
7. Demonstrate a permitted read and a denied ungranted operation with the test principal where allocated. Record the precise database scope and identity used.
8. Review the recovery plan. With an allocated test target, restore a training backup and measure data loss and elapsed time; otherwise explicitly mark the restore unperformed. Compare to the RPO/RTO requirement.
9. Submit the access matrix, secret-handling review and recovery evidence; clean up only the trainer-approved synthetic assets.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a data-access and recovery exception report with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.

- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.